T5 Análisis discriminante cuadrático y lineal

Índice

1. Introducción
2. Análisis discriminante cuadrático (QDA)
3. Análisis discriminante lineal (LDA)

1 Introducción

Objetivo: introducir quadratic discriminant analysis (QDA) y linear discriminant analysis (LDA)

Clasificadores generativos: expresan **posteriors** en función de **priors** y **densidades condicionales** de las clases

$$p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) = \frac{p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta})\, p(y = c, \boldsymbol{\theta})}{\sum_{c'} p(\boldsymbol{x} \mid y = c', \boldsymbol{\theta})\, p(y = c', \boldsymbol{\theta})} \propto p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta})\, p(y = c, \boldsymbol{\theta})$$

Ajuste de generativos: **maximización de la log-verosimilitud conjunta**, $\mathcal{L}(\boldsymbol{\theta}) = \sum_n \log p(\boldsymbol{x}_n, y_n \mid \boldsymbol{\theta})$

- **Sencillez:** los parámetros de cada clase suelen ajustarse separadamente, mediante promediado de datos de la clase
- **Flexibilidad:** pueden extenderse a nuevas clases sin necesidad de re-entrenar parámetros ya entrenados
- **Datos perdidos:** pueden tratarse fácilmente gracias al modelado de condicionales
- **Robustez:** frente a características degeneradas, también gracias al modelado de condicionales
- **Aprendizaje semi-supervisado:** facilitan el aprovechamiento de datos no etiquetados en aprendizaje

Análisis discriminante cuadrático (QDA): modela las condicionales de clase con Gaussianas independientes

Análisis discriminante lineal (LDA): modela las condicionales de clase con Gaussianas de Σ común

3 Análisis discriminante cuadrático (QDA)

Análisis discriminante Gaussiano (GDA): clasificador generativo de densidades condicionales Gaussianas

$$p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) \propto p(y = c, \boldsymbol{\theta})\, p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta}) = \pi_c \mathcal{N}_D(\boldsymbol{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

Función discriminante (en GDA): log-posterior de la clase vista como función de la entrada $\boldsymbol{x}$

$$\begin{aligned}\log p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) &\propto \log \pi_c + \log \mathcal{N}_D(\boldsymbol{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \\ &= \log \pi_c - \frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}_c| - \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}_c^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_c)\end{aligned}$$

Análisis discriminante cuadrático (QDA): GDA expresado en términos de discriminantes; cuadráticas con $\boldsymbol{x}$

$$\begin{aligned}\log p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) &\propto \boldsymbol{x}^t \mathbf{A}_c \boldsymbol{x} + \boldsymbol{\beta}_c^t \boldsymbol{x} + \gamma_c + \kappa \\ \mathbf{A}_c &= -\frac{1}{2}\boldsymbol{\Sigma}_c^{-1} \\ \boldsymbol{\beta}_c &= \boldsymbol{\Sigma}_c^{-1}\boldsymbol{\mu}_c \\ \gamma_c &= \log \pi_c - \frac{1}{2}\log|\boldsymbol{\Sigma}_c| - \frac{1}{2}\boldsymbol{\mu}_c^t \boldsymbol{\Sigma}_c^{-1}\boldsymbol{\mu}_c \\ \kappa &= -\frac{D}{2}\log(2\pi) \quad \text{(puede ignorarse porque no depende de } c\text{)}\end{aligned}$$

QDA diagonal: asume matrices de covarianzas diagonales; de $O(CD^2)$ a $O(CD)$ parámetros

Ejemplo: $C = 3,\ D = 2,\ \pi_1 = \pi_2 = \pi_3 = 1/3$ (podemos ignorar $\log \pi_c$ en γ_c)

$$\boldsymbol{\mu}_1 = (0,0)^t \qquad \boldsymbol{\mu}_2 = (0,4)^t \qquad \boldsymbol{\mu}_3 = (4,4)^t$$

$$\boldsymbol{\Sigma}_1 = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix} \qquad \boldsymbol{\Sigma}_2 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \qquad \boldsymbol{\Sigma}_3 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\boldsymbol{\Sigma}_1^{-1} = \begin{pmatrix} 2/7 & -1/7 \\ -1/7 & 4/7 \end{pmatrix} \qquad \boldsymbol{\Sigma}_2^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1 \end{pmatrix} \qquad \boldsymbol{\Sigma}_3^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

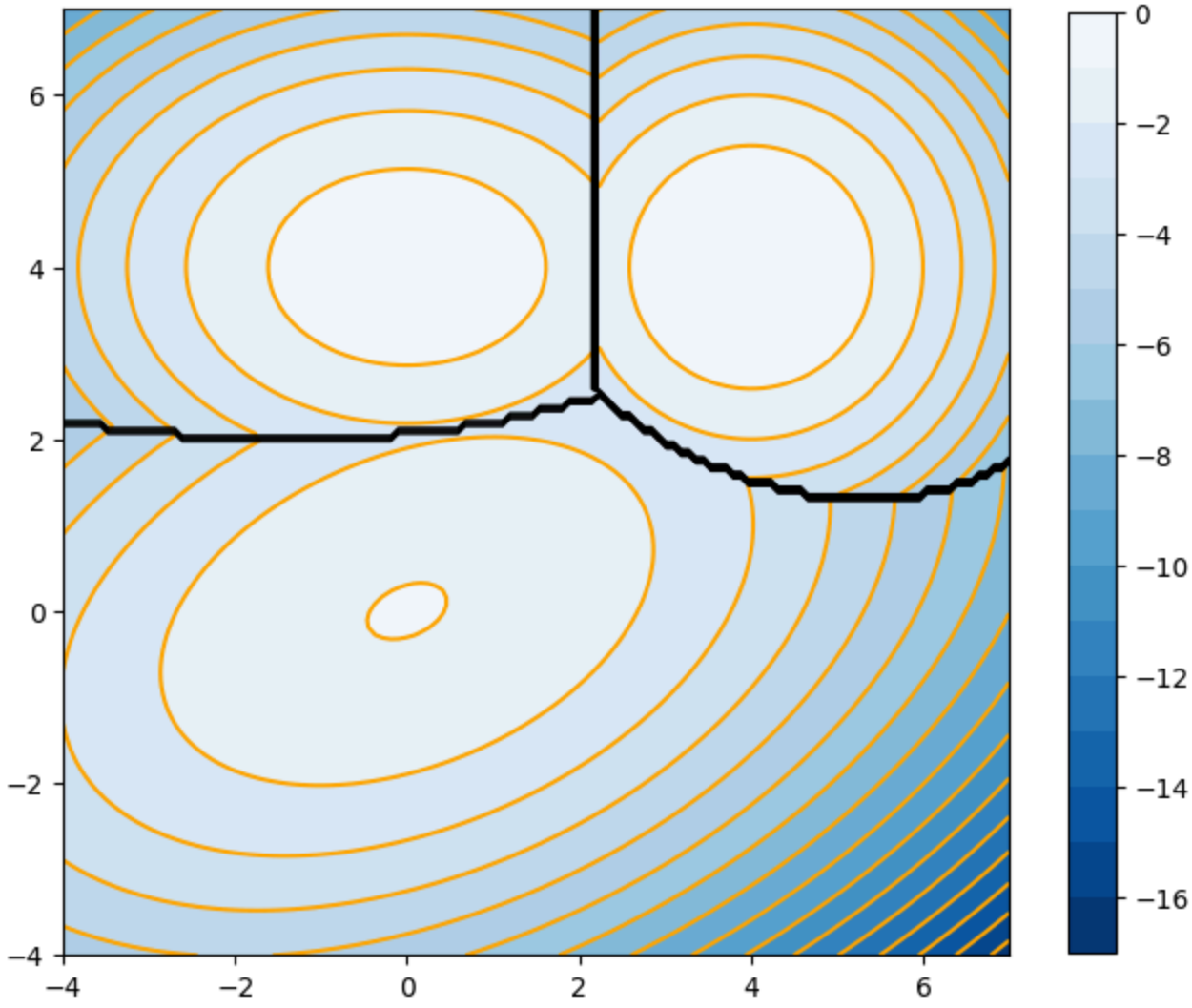
$$\mathbf{A}_1 = \begin{pmatrix} -1/7 & 1/14 \\ 1/14 & -2/7 \end{pmatrix} \qquad \mathbf{A}_2 = \begin{pmatrix} -1/4 & 0 \\ 0 & -1/2 \end{pmatrix} \qquad \mathbf{A}_3 = \begin{pmatrix} -1/2 & 0 \\ 0 & -1/2 \end{pmatrix}$$

$$\boldsymbol{\beta}_1 = (0,0)^t \qquad \boldsymbol{\beta}_2 = (0,4)^t \qquad \boldsymbol{\beta}_3 = (4,4)^t$$

$$\gamma_1 = -\frac{1}{2}\log 7 = -0.973 \qquad \gamma_2 = -\frac{1}{2}\log 2 - 8 = -8.3466 \qquad \gamma_3 = -16$$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
A1, b1, c1 = np.array([ [-1/7, 1/14], [1/14,  -2/7] ]), np.array([0, 0]), -0.973
A2, b2, c2 = np.array([ [-1/4,    0], [   0,  -1/2] ]), np.array([0, 4]), -8.3466
A3, b3, c3 = np.array([ [-1/2,    0], [   0,  -1/2] ]), np.array([4, 4]), -16
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 =  lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 =  lambda x: x.T @ A2 @ x + b2 @ x + c2
p3 =  lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x);
```

In [ ]:
```
fig, ax = plt.subplots(1, 1, figsize=(8, 8)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyles='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Log-verosimilitud conjunta: $\boldsymbol{\theta}' = \{\boldsymbol{\theta}_c\},\ \boldsymbol{\theta}_c = (\boldsymbol{\mu}_c, \mathrm{vec}(\boldsymbol{\Sigma}_c))^t$

$$\log p(\mathcal{D} \mid \boldsymbol{\theta}) = \sum_c N_c \log \pi_c + \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

Ajuste de $\boldsymbol{\theta}'$: $\hat{\boldsymbol{\theta}}'$ puede hallarse mediante maximización separada en cada $\boldsymbol{\theta}_c$ (con restricciones)

$$\hat{\boldsymbol{\theta}}' = \operatorname*{argmax}_{\boldsymbol{\theta}' \in \mathcal{C}_{\boldsymbol{\theta}'}} \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

$$\Updownarrow$$

$$\hat{\boldsymbol{\theta}}_c = \operatorname*{argmax}_{\boldsymbol{\theta}_c \in \mathcal{C}_{\boldsymbol{\theta}_c}} \sum_{n:y_n=c} \log \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{para todo } c$$

Ajuste de $\boldsymbol{\theta}'_c$: $\hat{\boldsymbol{\mu}}_c$ y $\hat{\boldsymbol{\Sigma}}_c$ se hallan con los datos de la clase c como el MLE de una única Gaussiana

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \boldsymbol{x}_n$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} (\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^t$$

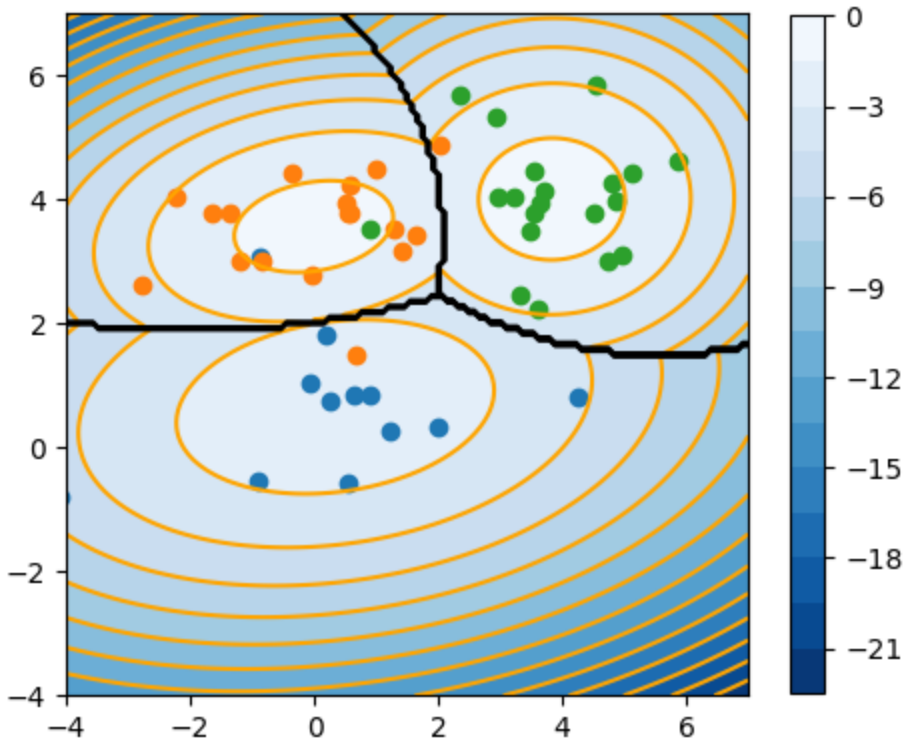
Ejemplo (cont.): $C = 3,\ D = 2,\ \pi_1 = \pi_2 = \pi_3 = 1/3$

$$\boldsymbol{\mu}_1 = (0,0)^t \qquad \boldsymbol{\mu}_2 = (0,4)^t \qquad \boldsymbol{\mu}_3 = (4,4)^t$$
$$\boldsymbol{\Sigma}_1 = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix} \qquad \boldsymbol{\Sigma}_2 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \qquad \boldsymbol{\Sigma}_3 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

In [ ]:
```python
import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import multinomial, multivariate_normal
N = 50 # >=3 para tener al menos un dato por clase; >>3 para evitar matrices singulares
pi1 = pi2 = pi3 = 1/3
yy = multinomial(1, [pi1, pi2, pi3]).rvs(N - 3)
N1 = yy[yy[:, 0] == 1].shape[0] + 1
N2 = yy[yy[:, 1] == 1].shape[0] + 1
N3 = N - N1 - N2
hpi1 = N1/N; hpi2 = N2/N; hpi3 = N3/N
xxy1 = multivariate_normal([0, 0], [[4, 1], [1, 2]]).rvs(N1)
m1 = xxy1.mean(axis=0); S1 = np.cov(xxy1, rowvar=False, bias=True); iS1 = np.linalg.inv(S1)
xxy2 = multivariate_normal([0, 4], [[2, 0], [0, 1]]).rvs(N2)
m2 = xxy2.mean(axis=0); S2 = np.cov(xxy2, rowvar=False, bias=True); iS2 = np.linalg.inv(S2)
xxy3 = multivariate_normal([4, 4], np.eye(2)).rvs(N3)
m3 = xxy3.mean(axis=0); S3 = np.cov(xxy3, rowvar=False, bias=True); iS3 = np.linalg.inv(S3)
print("Medias: ", m1, m2, m3, "\nSigmas:\n", S1, "\n", S2, "\n", S3)
```

```
Medias:  [0.33956314 0.65842643] [-0.00803341  3.56576981] [3.83229691 4.00927197]
Sigmas:
 [[3.53223658 0.36163145]
 [0.36163145 1.06158044]]
 [[1.82163006 0.21941638]
 [0.21941638 0.60543893]]
 [[1.20967908 0.00839126]
 [0.00839126 0.83738662]]
```

In [ ]:
```
A1, b1, c1 = -0.5 * iS1, iS1 @ m1, np.log(hpi1) - 0.5 * np.log(np.linalg.det(S1)) - 0.5 * m1.T @ iS1 @ m1
A2, b2, c2 = -0.5 * iS2, iS2 @ m2, np.log(hpi2) - 0.5 * np.log(np.linalg.det(S2)) - 0.5 * m2.T @ iS2 @ m2
A3, b3, c3 = -0.5 * iS3, iS3 @ m3, np.log(hpi3) - 0.5 * np.log(np.linalg.det(S3)) - 0.5 * m3.T @ iS3 @ m3
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]; p1 =  lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 =  lambda x: x.T @ A2 @ x + b2 @ x + c2; p3 =  lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyles='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8); ax.set_xlim(-4, 7); ax.set_ylim(-4, 7)
plt.scatter(xxy1[:, 0], xxy1[:, 1]); plt.scatter(xxy2[:, 0], xxy2[:, 1])
plt.scatter(xxy3[:, 0], xxy3[:, 1]);
```



4 Análisis discriminante lineal (LDA)

Análisis discriminante lineal (LDA): GDA con Gaussianas de **matriz de covarianza común,** $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$

$$\log p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) \propto \log \pi_c + \log \mathcal{N}_D(\boldsymbol{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

Discriminantes lineales con $\boldsymbol{x}$: al igual que κ, el término cuadrático puede ignorarse porque no depende de c

$$\begin{aligned}
\log p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) &\propto \log \pi_c - \frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}| - \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_c) \\
&= \boldsymbol{x}^t \mathbf{A} \boldsymbol{x} + \boldsymbol{\beta}_c^t \boldsymbol{x} + \gamma_c + \kappa \\
\mathbf{A} &= -\frac{1}{2}\boldsymbol{\Sigma}^{-1} \\
\boldsymbol{\beta}_c &= \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c \\
\gamma_c &= \log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^t \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c \\
\kappa &= -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}|
\end{aligned}$$

LDA diagonal: asume $\boldsymbol{\Sigma}$ diagonal; de $O(CD + D^2)$ a $O(CD)$ parámetros

Ejemplo: $C = 3,\ D = 2,\ \pi_1 = \pi_2 = \pi_3 = 1/3$ (podemos ignorar $\log \pi_c$ en γ_c)

$$\boldsymbol{\mu}_1 = (0,0)^t \qquad \boldsymbol{\mu}_2 = (0,4)^t \qquad \boldsymbol{\mu}_3 = (4,4)^t$$

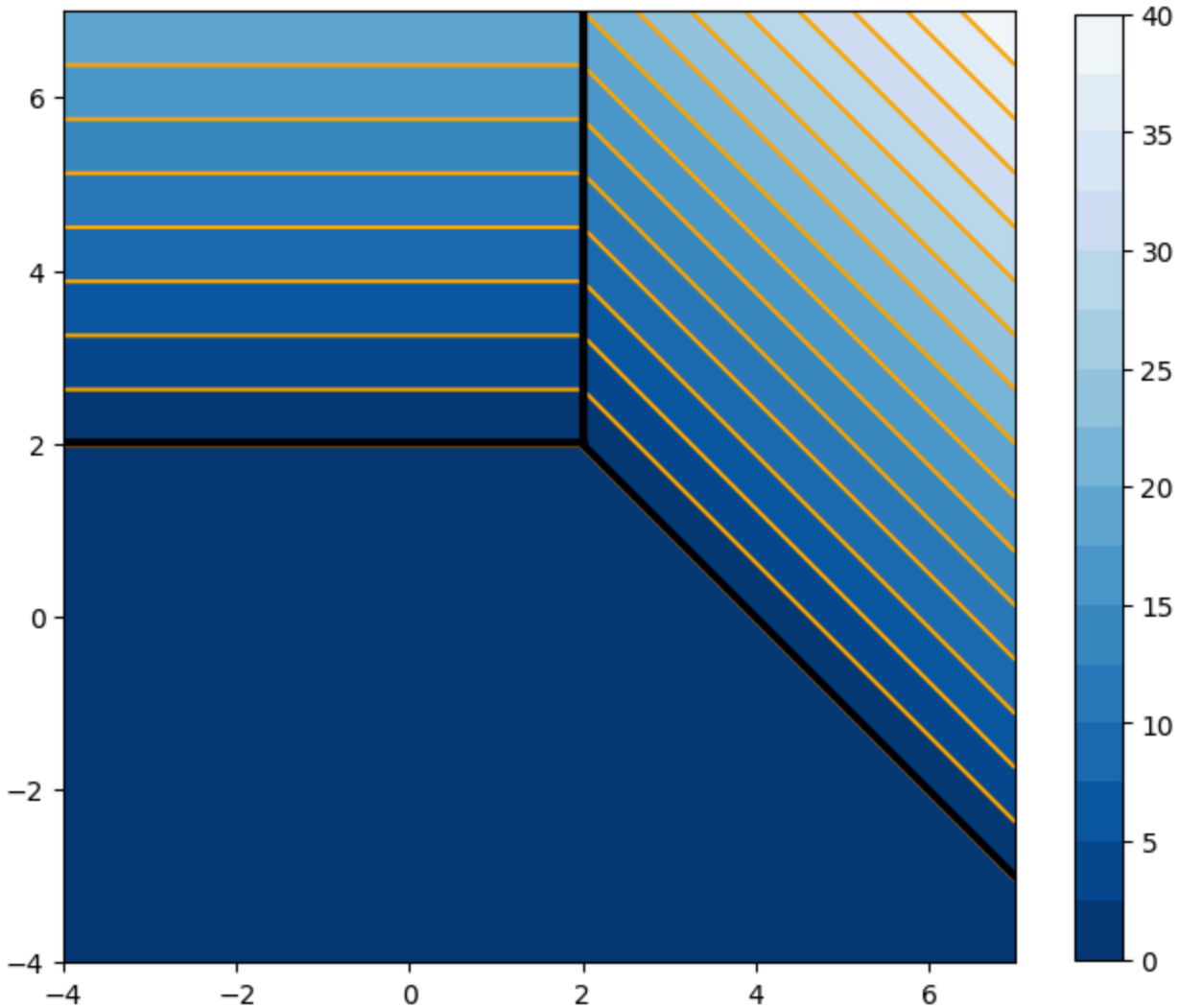
$$\boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\boldsymbol{\beta}_1 = (0,0)^t \qquad \boldsymbol{\beta}_2 = (0,4)^t \qquad \boldsymbol{\beta}_3 = (4,4)^t$$

$$\gamma_1 = 0 \qquad \gamma_2 = -8 \qquad \gamma_3 = -16$$

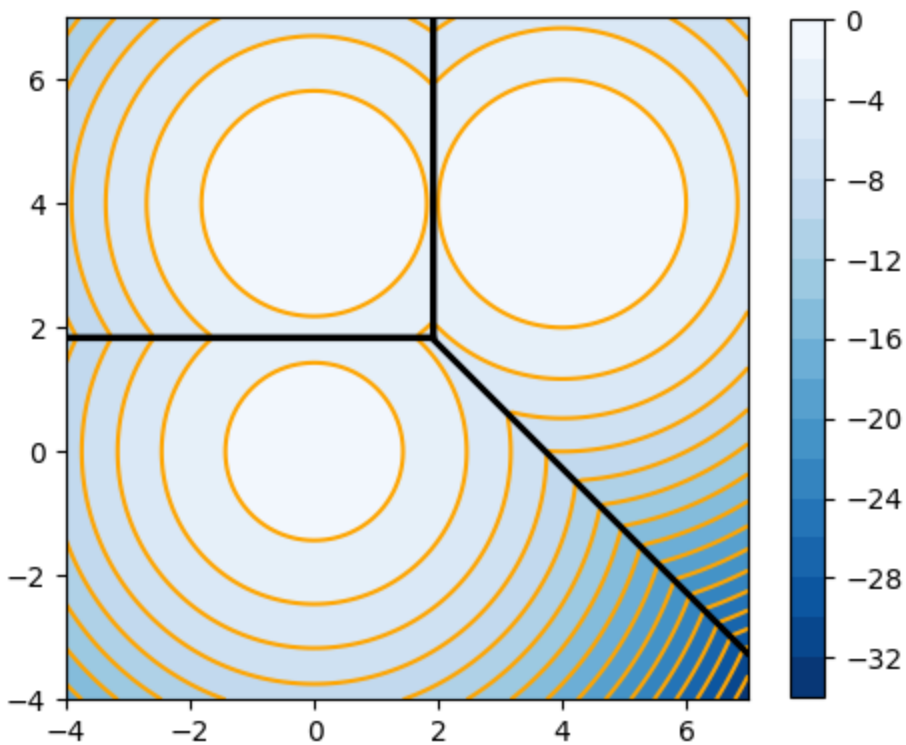
In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
b1, c1 = np.array([0, 0]), 0
b2, c2 = np.array([0, 4]), -8
b3, c3 = np.array([4, 4]), -16
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 =  lambda x: b1 @ x + c1
p2 =  lambda x: b2 @ x + c2
p3 =  lambda x: b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x)
```

In [ ]:
```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyles='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Los conjuntos de nivel típicos de las Gaussianas pueden obtenerse añadiendo el término cuadrático común a las discriminantes, si bien el mismo es irrelevante a efectos de regiones y fronteras de decisión:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
A1, b1, c1 = np.array([ [-1/2,    0], [   0,  -1/2] ]), np.array([0, 0]), -0.973
A2, b2, c2 = np.array([ [-1/2,    0], [   0,  -1/2] ]), np.array([0, 4]), -8.3466
A3, b3, c3 = np.array([ [-1/2,    0], [   0,  -1/2] ]), np.array([4, 4]), -16
R = np.linspace(-4, 7, 128); x1, x2 = np.meshgrid(R, R); x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 =  lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 =  lambda x: x.T @ A2 @ x + b2 @ x + c2
p3 =  lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyles='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Log-verosimilitud conjunta: $\boldsymbol{\theta}' = (\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_C, \mathrm{vec}(\boldsymbol{\Sigma}))^t$

$$\log p(\mathcal{D} \mid \boldsymbol{\theta}) = \sum_c N_c \log \pi_c + \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

Ajuste de $\boldsymbol{\theta}'$: no puede hacerse clase a clase, separadamente, porque las condicionales comparten una misma $\boldsymbol{\Sigma}$

$$\hat{\boldsymbol{\theta}}' = \operatorname*{argmax}_{\boldsymbol{\theta}' \in \mathcal{C}_{\boldsymbol{\theta}'}} \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

No obstante, la única diferencia respecto a QDA es que $\hat{\boldsymbol{\Sigma}}$ se obtiene con todos los datos:

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \boldsymbol{x}_n$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_c \sum_{n:y_n=c} (\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^t$$

Ejemplo (cont.): $C = 3,\ D = 2,\ \pi_1 = \pi_2 = \pi_3 = 1/3$

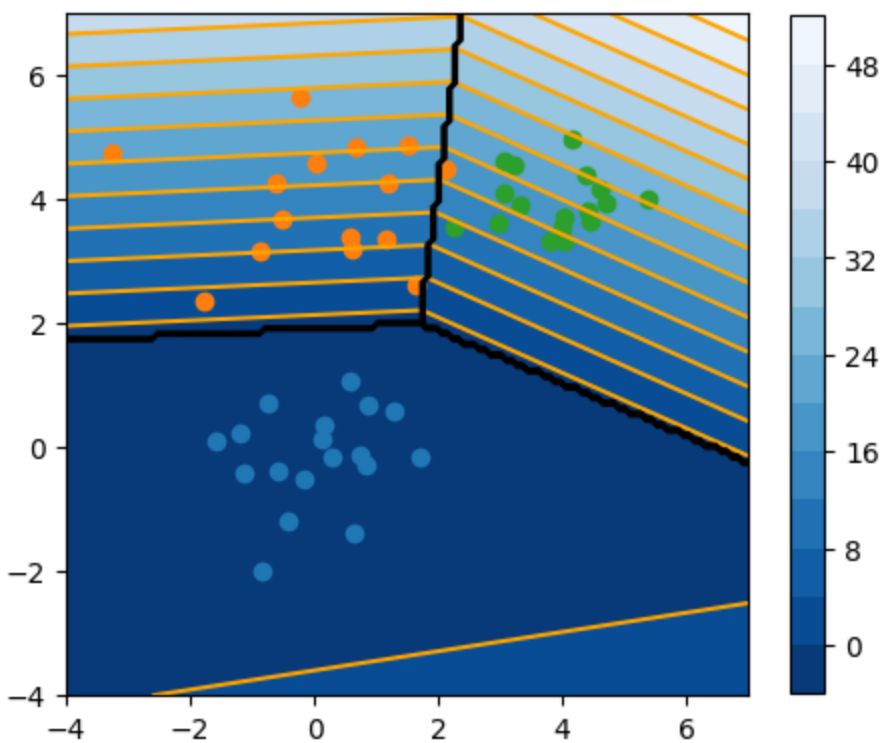
$$\boldsymbol{\mu}_1 = (0,0)^t \qquad \boldsymbol{\mu}_2 = (0,4)^t \qquad \boldsymbol{\mu}_3 = (4,4)^t$$

$$\boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

In [ ]:
```python
import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import multinomial, multivariate_normal
N = 50 # >=3 para tener al menos un dato por clase; >>3 para evitar matrices singulares
pi1 = pi2 = pi3 = 1/3
yy = multinomial(1, [pi1, pi2, pi3]).rvs(N - 3)
N1 = yy[yy[:, 0] == 1].shape[0] + 1
N2 = yy[yy[:, 1] == 1].shape[0] + 1
N3 = N - N1 - N2
hpi1 = N1/N; hpi2 = N2/N; hpi3 = N3/N
xxy1 = multivariate_normal([0, 0], np.eye(2)).rvs(N1)
m1 = xxy1.mean(axis=0); S1 = np.cov(xxy1, rowvar=False, bias=True)
xxy2 = multivariate_normal([0, 4], np.eye(2)).rvs(N2)
m2 = xxy2.mean(axis=0); S2 = np.cov(xxy2, rowvar=False, bias=True)
xxy3 = multivariate_normal([4, 4], np.eye(2)).rvs(N3)
m3 = xxy3.mean(axis=0); S3 = np.cov(xxy3, rowvar=False, bias=True)
S = hpi1 * S1 + hpi2 * S2 + hpi3 * S3; iS = np.linalg.inv(S)
print("Medias: ", m1, m2, m3, "\nSigma:\n", S)
```

```
Medias:  [ 0.02852604 -0.14860485] [0.14946012 3.96905472] [3.87641211 3.95226088]
Sigma:
 [[1.06255583 0.06651106]
 [0.06651106 0.52196998]]
```

In [ ]:
```
b1, c1 = iS @ m1, np.log(hpi1) - 0.5 * m1.T @ iS @ m1
b2, c2 = iS @ m2, np.log(hpi2) - 0.5 * m2.T @ iS @ m2
b3, c3 = iS @ m3, np.log(hpi3) - 0.5 * m3.T @ iS @ m3
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 =  lambda x: b1 @ x + c1; p2 =  lambda x: b2 @ x + c2; p3 =  lambda x: b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyles='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyles='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8); ax.set_xlim(-4, 7); ax.set_ylim(-4, 7)
plt.scatter(xxy1[:, 0], xxy1[:, 1]); plt.scatter(xxy2[:, 0], xxy2[:, 1])
plt.scatter(xxy3[:, 0], xxy3[:, 1]);
```



Clasificador por el centroide más próximo (NCM): regla de decisión MAP para LDA con priors idénticos,

$$
\begin{aligned}
\hat{y}(\boldsymbol{x}) &= \operatorname*{argmax}_c \ \log p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) \\
&= \operatorname*{argmax}_c \ \log p(\boldsymbol{x} \mid y = c, \boldsymbol{\theta}) && \text{(asumiendo priors idénticos)} \\
&= \operatorname*{argmax}_c \ -\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_c) \\
&= \operatorname*{argmin}_c \ (\boldsymbol{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_c) \\
&= \operatorname*{argmin}_c \ \|\boldsymbol{x} - \boldsymbol{\mu}_c\|^2_{\mathbf{W}} && \text{(con } \boldsymbol{\Sigma}^{-1} = \mathbf{W}^t\mathbf{W},\ \|\boldsymbol{z}\|^2_{\mathbf{W}} = \boldsymbol{z}^t\mathbf{W}^t\mathbf{W}\boldsymbol{z}\text{)}
\end{aligned}
$$

Motivación: aparte de su sencillez y reducido coste computacional, puede usarse en **one-shot learning** de nuevas clases ya que basta añadir un prototipo etiquetado $\boldsymbol{\mu}_c$ por cada clase c nueva que se quiera incluir

Aprendizaje de la métrica: $\mathbf{W}$ puede ser una raíz de $\boldsymbol{\Sigma}^{-1}$ o aprenderse con algún otro criterio